

FEEL BEFORE YOU BUILD

Hands-on prototyping with actuators · 2 hours

TLDR

Two hours finding out what actuators feel like, and leaving able to drive one from your own code.

Get a board and run task 01. Something spins.

Change two numbers, upload again. That loop is the workshop.

Pick up the actuators you are not driving.

Work through as many of the five tasks as you get to.

Build one signal someone else can read without looking.

WHY WE ARE HERE

You can read a datasheet. You have never held one of these.

Most projects end up on the screen and the speaker

- Not because touch was wrong
- Because nobody knew the lab has 117 coin motors

Today: decide it feels cheap, pick something else

- Better now than in week 46

HOW TODAY WORKS

A board each. You build the circuits yourself

- Every part, every wire, from an empty breadboard

Four tasks, each one wire more than the last

- Piezo, then a touch pad, then both, then a stepper

Actuators are on the tables at the front

- Go and feel the ones you did not build

Finish by building one signal someone else can read

RUN SHEET

Times are offsets from the start. The only hard checkpoint is 0:25: everyone has task 01 running.

0:00	10 min	Two motors, same message. Which felt urgent?
0:10	15 min	Task 00: piezo. Two wires, everyone
0:25	20 min	Task 04: one wire and foil. Watch the drift
0:45	20 min	Task 05: join them. Touch in, tick out
1:05	20 min	Task 07: stepper, if you want it
1:25	10 min	Round-the-room: what surprised you
1:35	20 min	Blind test on task 05
1:55	5 min	Pack down

THE TASKS, IN BUILD ORDER

Each one adds a single new thing to the circuit before it.

- 00 Make it tick piezo 2 wires**
- 04 Make it notice you a wire and foil 1 wire**
- 05 Make it answer both of the above 3 wires**
- 07 Make it turn stepper 6 wires**

Four tasks, and the first three need no driver at all.

Tasks 01 to 03 are in the repo if you want the motor,
the solenoid or the Peltier tile. We will not do them today.

HOW THE TASK SKETCHES WORK

Wire it from the diagram at the top, then upload. All of them: github.com/labtools-au/multimodal-actuator-workshop

A PINS block at the very top

- Every leg, every wire. Build from that, not from memory

A CHANGE ME block below it

- Two or three numbers. Edit, upload, feel the difference

A THINGS TO TRY list at the bottom

- Four experiments. The last is the interesting one

PIN MAP

01	spin	D9	PWM	1k to PN2222 base, motor on collector
02	tap	D6		MOSFET gate, own supply, grounds joined
03	warm	D3 + D5	PWM	H-bridge low and high, bench supply
04	sense	A0		bare wire to foil. That is all of it
05	answer	A0 + D9		the pad, plus any actuator
07	stepper	D8 D9 D10 D11		ULN2003 IN1 to IN4

Only D3, D5, D6, D9, D10, D11 do PWM. Stepper pins go into the constructor OUT of order: 8, 10, 9, 11.

WHAT YOU GET IN THE BASE KIT

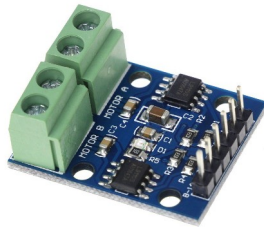
One set per pair, collected before you start task 01. Below: the three parts people always come back for.

Board, breadboard, jumper wires, resistors: on the tables.

Not in the drawers, so take what you need.



Breadboard supply, 8A. Anything past an LED.

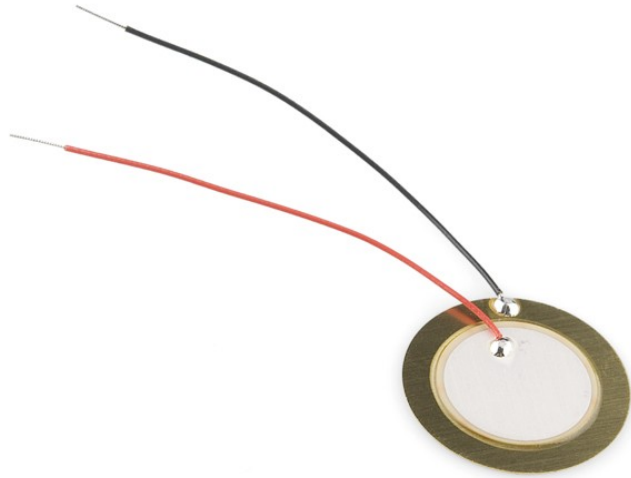


L9110S H-bridge, 2C. Reversing, and the Peltier.



PAM8403 amp, 11F. A transducer needs one.

PIEZO ELEMENT



A crystal that flexes when you put current across it. Near-instant, almost no power, but shallow.

Feels like: a tick, not a thump.

USED FOR

Clicks and confirmations. Anything where lag would feel broken.

Drawer 6F | 69 in stock | two wires, no driver

CAPACITIVE PAD



A wire taped behind foil, card or fabric. No sensor and no breakout board, which is why the box on the left is empty.

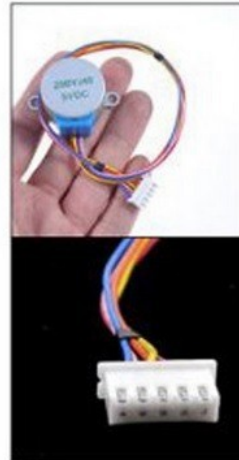
Feels like: nothing. That is the point. The surface stays plain.

USED FOR

Invisible controls in wood or fabric, waterproof panels.

No drawer, no part number, no cost | one wire | keep it on USB

STEPPER MOTOR



Moves in fixed steps rather than continuously, so you always know where it is without a sensor.

Feels like: precise, and audibly clicky.

USED FOR

Slow accurate motion. Dials, sliders, anything positioned.

2A | 44 | ULN2003 | 2048 steps/rev | own 5V supply

WHAT ELSE THE LAB HAS

You are not limited to today's three. All of this is in the drawers, and the repo has working code for the first three.

Coin motor **a buzz you cannot make gentle and fast**

- 1E | 117 in stock | needs a transistor

Solenoid **a person tapping you, not a signal**

- 3E | 24 | 1.1 A, needs its own supply

Peltier tile **heat and cold, several seconds to be sure**

- 4C | 60 | needs an H bridge

Surface transducer turns any surface into a speaker

- 7F | 48 | needs an amplifier

Servo **holds an angle, with force behind it**



AARHUS
UNIVERSITY

DEPARTMENT OF COMPUTERS

- 2F | 80 | Servo.h, own 5V

MULTIMODAL INTERACTION
PROTOTYPING WORKSHOP

GUSTAV SIMONSEN
TEACHING ASSISTANT



FREESTYLE, WITHIN THREE LIMITS

You do not have to do the tasks in order, or finish them. Playing properly with two beats rushing all five.

If you want to drive something not in front of you, go get it.

The three things that are not negotiable

- Solenoid and Peltier get their own supply, never USB
- The Peltier heatsink goes on before it is switched on
- No motor straight from a pin. 40 mA limit, it wants 75

THE ONE THAT SURPRISES PEOPLE

Touch input for the price of a wire. The thinking is where it costs you.

Threshold is relative, never absolute

- Readings drift with humidity and mains hum
- Fixed threshold works at 9am, fails at 1pm

Baseline only learns while untouched

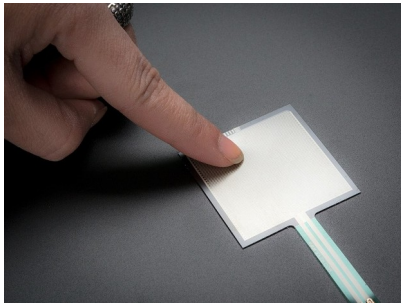
- Or a long press teaches it that a finger is normal

INPUTS, AND PRIOR ART

INPUTS WORTH KNOWING ABOUT

One table with a sign. Wander over between tasks.

- Capacitive touch. A wire. That is task 04
- GSR, drawer 0C5. Exactly one in the lab, so ask first



Force (FSR), 4A. 46 tiny, 33 square. Calibrate each one.



Flex, 4B. Only 8 left, so share them.



Pulse, 9D. 9 in stock. Steady at rest, useless once moving.

YOU ALREADY WRITE CODE. WHY COME?

Fair question. Three honest answers.

You cannot Google what something feels like

- No datasheet says a knock reads as a person, a buzz as a machine

The parts are free and already here

- Borrowing beats ordering. Nothing to buy, nothing to wait for

Your devices hide sensors you may not be able to reach

- Find out today which open up, and which are locked

REVERSE ENGINEER WHAT YOU ALREADY OWN

The trackers in your pocket and on your wrist. What opens up, and what does not.

BLE heart rate strap: readable from a web page

- Web Bluetooth, standard GATT. Chrome and Edge only

Most Garmin watches: the same, with no app

- Turn on Broadcast Heart Rate and it acts as a strap

Apple Watch: nothing live, whatever you write

- HealthKit needs a watchOS app in Swift

Garmin history: Connect IQ, or the API afterwards

- Good for analysis. Useless for reacting in real time

WHAT A PHONE WILL AND WILL NOT GIVE YOU

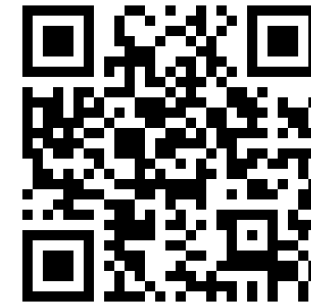
Open sensors.chomskylab.dk on your own phone right now.

Works in a web page

- Accelerometer and gyroscope, microphone, camera
- Pointer pressure, though a trackpad reports only 0 or 0.5

Does not, whatever the tutorial says

- Phone vibration on iOS. WebKit has never shipped it, at any version, and Chrome on iOS is Safari underneath



sensors.chomskylab.dk

So "the phone buzzes" works on no iPhone in this room.

- Check the feature, not the tutorial, before it is in a plan.

TWO RIGS THAT ALREADY EXIST

Built here, by students at roughly your stage.

Moving screen, Arduino Uno

- Breathes when idle. Retreats when touched
- Easing and idle motion do the work. A dozen lines

Instrumented sock, ESP32

- Six force sensors under a foot, batched over Wi-Fi
- Calibrate per sensor. Batch your writes

BUILD ONE SIGNAL

THE BRIEF

30 minutes. Not long enough to be precious about it.

One actuator, whichever you built

- Three messages: arrived, something wrong, finished

Vary rhythm above all

- Piezo: rhythm is all you have. Motor: strength too

Blind test: they name all three

- Note which two got confused, and why

THE CONSTRAINT THAT MATTERS

The recipient cannot look at the device.

If it only works while someone watches a screen,
you built a visual interface with a motor glued to it.

BEFORE YOU CALL IT DONE

Hand your device to another group with the screen turned away.

- They can tell all three messages apart
- It does not fire continuously when held
- You know which two got confused, and why

What you actually leave with

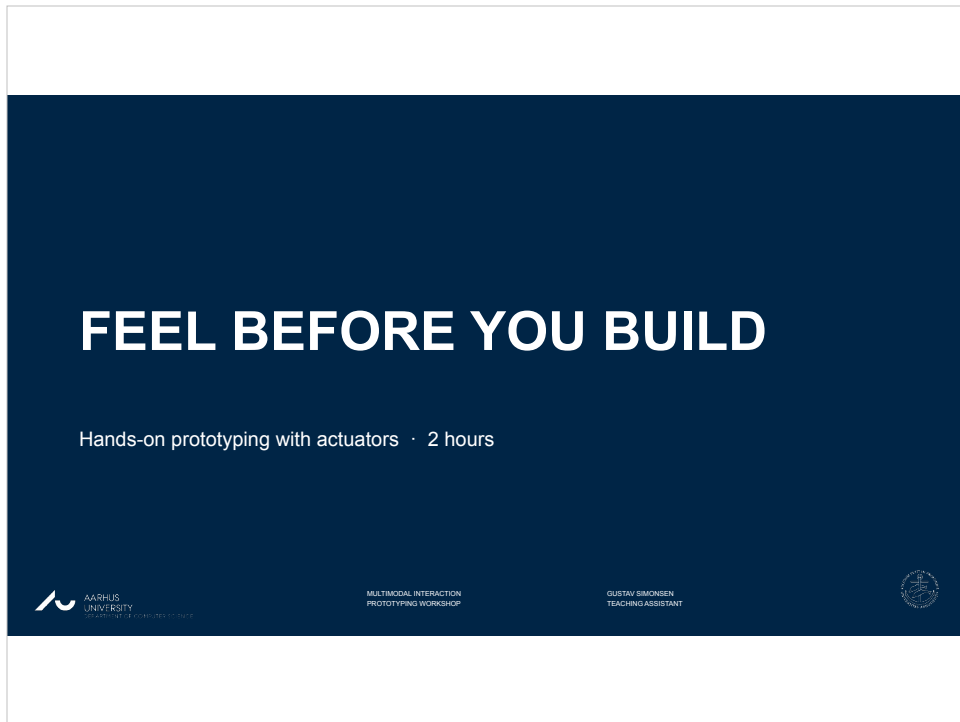
- What things feel like, how to drive one, and which to pick
- That last one is the expensive mistake this prevents

GO AND TOUCH THINGS

Everything is open: code, slides and session plan



[github.com/labtools-au/
multimodal-actuator-workshop](https://github.com/labtools-au/multimodal-actuator-workshop)



0:00. Start here, and do not read the slide.

Open with the demo, not with talking. Have the coin motor and the solenoid both wired and running before anyone sits down.

Buzz a phone-style notification on the motor. Then send the same message on the solenoid. Ask which felt more urgent. Let them answer. Nobody needs theory for that, and it is the argument for the whole session.

TLDR

Two hours finding out what actuators feel like, and leaving able to drive one from your own code.

Get a board and run task 01. Something spins.

Change two numbers, upload again. That loop is the workshop.

Pick up the actuators you are not driving.

Work through as many of the five tasks as you get to.

Build one signal someone else can read without looking.



Fifteen seconds. It is a map, not a talk.

Useful because half the room arrives expecting a lecture and needs to be told immediately that this is not one.

WHY WE ARE HERE

You can read a datasheet. You have never held one of these.

Most projects end up on the screen and the speaker

- Not because touch was wrong
- Because nobody knew the lab has 117 coin motors

Today: decide it feels cheap, pick something else

- Better now than in week 46



Say the full version out loud, the slide only carries the skeleton:

"You can read a datasheet. You have never held one of these. Today you put a vibration motor against your own wrist, decide it feels cheap, and pick something else. Better now than in week 46 with the demo three days out."

Every year projects end up on the screen and the speaker, and touch gets designed on paper then dropped. Two minutes, then move.

HOW TODAY WORKS

A board each. You build the circuits yourself

- Every part, every wire, from an empty breadboard

Four tasks, each one wire more than the last

- Piezo, then a touch pad, then both, then a stepper

Actuators are on the tables at the front

- Go and feel the ones you did not build

Finish by building one signal someone else can read



Set the shape clearly, because it is not a lecture and they will wait to be lectured at.

A board each, and they build the circuits themselves from an empty breadboard. Say that plainly, because a room of computer scientists will assume the hardware is someone else's problem and wait to be handed a working rig.

Two or three stations each, in rising difficulty. Nobody builds all six.

The ladder does the teaching: task 00 is two wires, task 04 is one, task 05

RUN SHEET

Times are offsets from the start. The only hard checkpoint is 0:25: everyone has task 01 running.

0:00	10 min	Two motors, same message. Which felt urgent?
0:10	15 min	Task 00: piezo. Two wires, everyone
0:25	20 min	Task 04: one wire and foil. Watch the drift
0:45	20 min	Task 05: join them. Touch in, tick out
1:05	20 min	Task 07: stepper, if you want it
1:25	10 min	Round-the-room: what surprised you
1:35	20 min	Blind test on task 05
1:55	5 min	Pack down



Show it, do not narrate it. Fifteen seconds.

Two hard checkpoints. At 0:20 every single person should have a piezo ticking, because that is two wires and if it is not working the problem is the board or the IDE, not the circuit. Fix those before anyone meets a transistor.

At 0:40 most people should have task 01 buzzing. If someone is still fighting it, pair them with a working group rather than leaving them to debug alone.

Walk the room during the 45 minute block.
Look for reversed diodes and

THE TASKS, IN BUILD ORDER

Each one adds a single new thing to the circuit before it.

- 00 Make it tick piezo 2 wires
- 04 Make it notice you a wire and foil 1 wire
- 05 Make it answer both of the above 3 wires
- 07 Make it turn stepper 6 wires

Four tasks, and the first three need no driver at all.

Tasks 01 to 03 are in the repo if you want the motor,
the solenoid or the Peltier tile. We will not do them today.



Ninety seconds on this slide, no more.

Read it as a ladder, because that is what it is.
The top three need no
transistor at all: piezo, foil pad, then the two
joined together. That is a
complete sense-and-respond loop built from
three wires, and they can have it
working inside twenty minutes.

The bottom three each add exactly one new
idea. Task 01 adds a transistor and
a flyback diode, because a motor draws 100
mA and a pin gives 40. Task 02 adds
a separate supply. Task 03 adds an H-bridge
for direction

HOW THE TASK SKETCHES WORK

Wire it from the diagram at the top, then upload. All of them: github.com/labtools-au/multimodal-actuator-workshop

A PINS block at the very top

- Every leg, every wire. Build from that, not from memory

A CHANGE ME block below it

- Two or three numbers. Edit, upload, feel the difference

A THINGS TO TRY list at the bottom

- Four experiments. The last is the interesting one



Say the format once here rather than repeating it per task.

PINS at the very top: every leg and every wire. That block is what they build from, and it is why nobody needs a schematic. Point at it explicitly.

CHANGE ME below that: two or three numbers, already set to something that works. THINGS TO TRY at the bottom: four experiments, easy to hard, and the last one usually asks for a rewrite rather than a tweak.

Upload problems in the order they will hit you:

PIN MAP

01	spin	D9	PWM	1k to PN2222 base, motor on collector
02	tap	D6		MOSFET gate, own supply, grounds joined
03	warm	D3 + D5	PWM	H-bridge low and high, bench supply
04	sense	A0		bare wire to foil. That is all of it
05	answer	A0 + D9		the pad, plus any actuator
07	stepper	D8 D9 D10 D11		ULN2003 IN1 to IN4

Only D3, D5, D6, D9, D10, D11 do PWM. Stepper pins go into the constructor OUT of order: 8, 10, 9, 11.



Reference, not a talking point. Ten seconds, or skip it entirely if the stations are already built.

It earns its place two ways: you use it while setting the room up, and it is the slide to jump back to when someone asks "which pin was the solenoid".

The bottom line is the one worth reading aloud if anyone touches the stepper: the pins go into the constructor OUT of order, 8, 10, 9, 11. Wired in numeric order it buzzes instead of turning and looks like a dead motor.

WHAT YOU GET IN THE BASE KIT

One set per pair, collected before you start task 01. Below: the three parts people always come back for.

Board, breadboard, jumper wires, resistors: on the tables.

Not in the drawers, so take what you need.



Breadboard supply, 8A. Anything past an LED.



L9110S H-bridge, 2C. Reversing, and the Pellier.



PAM8403 amp, 11F. A transducer needs one.

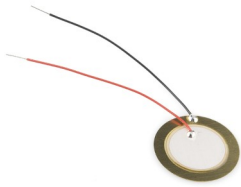


Hand this out before anything else, or you will spend the session fetching parts one at a time.

One set per pair. The three drawer numbers are the ones people actually come back and ask for: 8A breadboard supply, 2C H-bridge, 11F amplifier.

Jumper wires, breadboards and resistors are not tracked in the component database, so make sure there are enough on the tables before students arrive.

PIEZO ELEMENT



A crystal that flexes when you put current across it. Near-instant, almost no power, but shallow.

Feels like: a tick, not a thump.

USED FOR
Clicks and confirmations. Anything where lag would feel broken.

Drawer 6F | 69 in stock | two wires, no driver

AARHUS UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE

MULTIMODAL INTERACTION
PROTOTYPING WORKSHOP

GUSTAV SIMONSEN
TEACHING ASSISTANT



First actuator and first build, so keep it short and get them moving.

A crystal that flexes when you put current across it. Near-instant, almost no power, but shallow. A tick, not a thump.

Two wires and no driver, because a piezo is a capacitor and draws almost nothing. That is the honest reason it is first, and it is worth saying: when this works, their board, cable, port and IDE are all proven, so every later failure is in the circuit they built.

Tell them to press it flat against the table. It

CAPACITIVE PAD



PHOTO
Capacitive pad

A wire taped behind foil, card or fabric. No sensor and no breakout board, which is why the box on the left is empty.

Feels like: nothing. That is the point. The surface stays plain.

USED FOR

Invisible controls in wood or fabric, waterproof panels.

No drawer, no part number, no cost | one wire | keep it on USB



AARHUS
UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE

MULTIMODAL INTERACTION
PROTOTYPING WORKSHOP

GUSTAV SIMONSEN
TEACHING ASSISTANT



Hold up a wire and a piece of foil. That is genuinely the whole sensor.

This is the slide that changes what people think is possible, because it costs nothing and hides completely inside a prototype.

Two things to say now rather than when it bites. Keep the board on laptop USB: the sensing measures your body against the board's ground and that reference arrives through the mains earth, so on a charger or a battery it goes unstable or dead with nothing visibly wrong. And hands off the pad while it boots, or it

STEPPER MOTOR



Moves in fixed steps rather than continuously, so you always know where it is without a sensor.

Feels like: precise, and audibly clicky.

USED FOR

Slow accurate motion. Dials, sliders, anything positioned.

2A | 44 | ULN2003 | 2048 steps/rev | own 5V supply



The precision option, and the only actuator here that goes to a POSITION rather than a state. 2048 steps per revolution, no feedback sensor needed.

Two things will bite, so say both before they start.

The constructor does NOT take the pins in numeric order. Wire IN1 to IN4 on pins 8 to 11, then write `Stepper(2048, 8, 10, 9, 11)`. Get it wrong and it buzzes and jitters instead of turning, which looks exactly like a dead motor.

Give the driver its own 5V supply. Each coil

WHAT ELSE THE LAB HAS

You are not limited to today's three. All of this is in the drawers, and the repo has working code for the first three.

Coin motor **a buzz you cannot make gentle and fast**

- 1E | 117 in stock | needs a transistor

Solenoid **a person tapping you, not a signal**

- 3E | 24 | 1.1 A, needs its own supply

Peltier tile **heat and cold, several seconds to be sure**

- 4C | 60 | needs an H bridge

Surface transducer turns any surface into a speaker

- 7F | 48 | needs an amplifier



Servo **holds an angle, with force behind it**

- 2F | 80 | Servo.h, own 5V

PROTOTYPING WORKSHOP

GUSTAV SIMONSEN
TEACHING ASSISTANT



Ninety seconds, and do not read the list. The purpose is permission, not coverage.

They are building three things today. This slide says the drawers hold a lot more, so nobody designs a project around a coin motor just because it was the one on the slide.

Point out that tasks 01 to 03 are already in the repo with working code and wiring for the motor, the solenoid and the Peltier tile. They can pick those up in project weeks without starting from nothing.

FREESTYLE, WITHIN THREE LIMITS

You do not have to do the tasks in order, or finish them. Playing properly with two beats rushing all five.

If you want to drive something not in front of you, go get it.

The three things that are not negotiable

- Solenoid and Peltier get their own supply, never USB
- The Peltier heatsink goes on before it is switched on
- No motor straight from a pin. 40 mA limit, it wants 75



Two messages on one slide, and they pull in opposite directions on purpose. Say the permission warmly, then change tone for the limits.

Permission first: some students will not believe they are allowed to skip ahead or stop early unless told. The failure mode is a pair rushing all five badly instead of doing two properly.

Then the three hard limits, and here you should sound firm. Say the numbers out loud: a pin gives 40 mA, the motor wants about 75. It is not a rule they have to take on faith

THE ONE THAT SURPRISES PEOPLE

Touch input for the price of a wire. The thinking is where it costs you.

Threshold is relative, never absolute

- Readings drift with humidity and mains hum
- Fixed threshold works at 9am, fails at 1pm

Baseline only learns while untouched

- Or a long press teaches it that a finger is normal



Slow down here. This is the single idea most worth stealing from today, and it is worth interrupting the room for once most people have reached task 04.

Two things students get wrong:

1. A fixed threshold. Works in the morning, fails after lunch, because the reading drifts with humidity and mains hum.
2. Feeding the baseline buffer always. A long press then teaches the baseline that a finger is normal, and the touch silently stops working. The buffer must only learn while untouched

INPUTS, AND PRIOR ART



1:25. Pull the room back together.

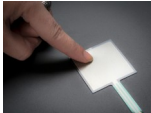
One sentence per group: what surprised you.
No slides, no laptops. Ten minutes.

Expect someone to report the Peltier delay. Let
that land, it is far more
persuasive from a peer than from you.

INPUTS WORTH KNOWING ABOUT

One table with a sign. Wander over between tasks.

- Capacitive touch. A wire. That is task 04
- GSR, drawer 0C5. Exactly one in the lab, so ask first



Force (FSR), 4A. 46 tiny, 33 square. Calibrate each one.



Flex, 4B. Only 8 left, so share them.



Pulse, 9D. 9 in stock. Steady at rest, useless once moving.



Mention, do not teach. These sit on one table with a sign.

The honest warnings matter more than the specs: PPG is useless once the hand moves, GSR drifts so only relative change works, FSRs need per-pad calibration.

Push the phone option hard. For a lot of projects it beats everything else on the table and needs no soldering at all.

YOU ALREADY WRITE CODE. WHY COME?

Fair question. Three honest answers.

You cannot Google what something feels like

- No datasheet says a knock reads as a person, a buzz as a machine

The parts are free and already here

- Borrowing beats ordering. Nothing to buy, nothing to wait for

Your devices hide sensors you may not be able to reach

- Find out today which open up, and which are locked



The slide that earns the room's attention, so do not rush it.

This audience can write the code already, and they know it. Saying so out loud buys credibility for the rest. The argument is not "you need to learn to program a motor", it is "you cannot Google what something feels like".

The second point is practical and lands with everyone: the parts are free and already in the building.

REVERSE ENGINEER WHAT YOU ALREADY OWN

The trackers in your pocket and on your wrist. What opens up, and what does not.

BLE heart rate strap: readable from a web page

- Web Bluetooth, standard GATT. Chrome and Edge only

Most Garmin watches: the same, with no app

- Turn on Broadcast Heart Rate and it acts as a strap

Apple Watch: nothing live, whatever you write

- HealthKit needs a watchOS app in Swift

Garmin history: Connect IQ, or the API afterwards

- Good for analysis. Useless for reacting in real time



The slide the tinkerers will care most about, and the one where being precise matters, because the internet is full of wrong answers.

The useful surprise: most Garmin watches broadcast heart rate over BLE natively. Turn on Broadcast Heart Rate, or start a Virtual Run, and the watch appears as an ordinary heart rate strap that Web Bluetooth can read from a web page. No Connect IQ app, no SDK, no API key.

Apple Watch will not. HealthKit needs a companion watchOS app in Swift, so

WHAT A PHONE WILL AND WILL NOT GIVE YOU

Open sensors.chomskylab.dk on your own phone right now.

Works in a web page

- Accelerometer and gyroscope, microphone, camera
- Pointer pressure, though a trackpad reports only 0 or 0.5

Does not, whatever the tutorial says

- Phone vibration on iOS. WebKit has never shipped it, at any version, and Chrome on iOS is Safari underneath



sensors.chomskylab.dk

So "the phone buzzes" works on no iPhone in this room.

- Check the feature, not the tutorial, before it is in a plan.



Have them open sensors.chomskylab.dk on their own phones while you talk. The split between what works and what does not lands far harder on their own device than on a slide.

The bottom half is the single most useful fact here, and the one people get wrong because a blog post said otherwise.

Say it plainly: iPhones do not vibrate from a web page. Not with a polyfill, not with a library, not in Chrome for iOS, which is Safari underneath.

The general lesson is bigger than vibration:

TWO RIGS THAT ALREADY EXIST

Built here, by students at roughly your stage.

Moving screen, Arduino Uno

- Breathes when idle. Retreats when touched
- Easing and idle motion do the work. A dozen lines

Instrumented sock, ESP32

- Six force sensors under a foot, batched over Wi-Fi
- Calibrate per sensor. Batch your writes

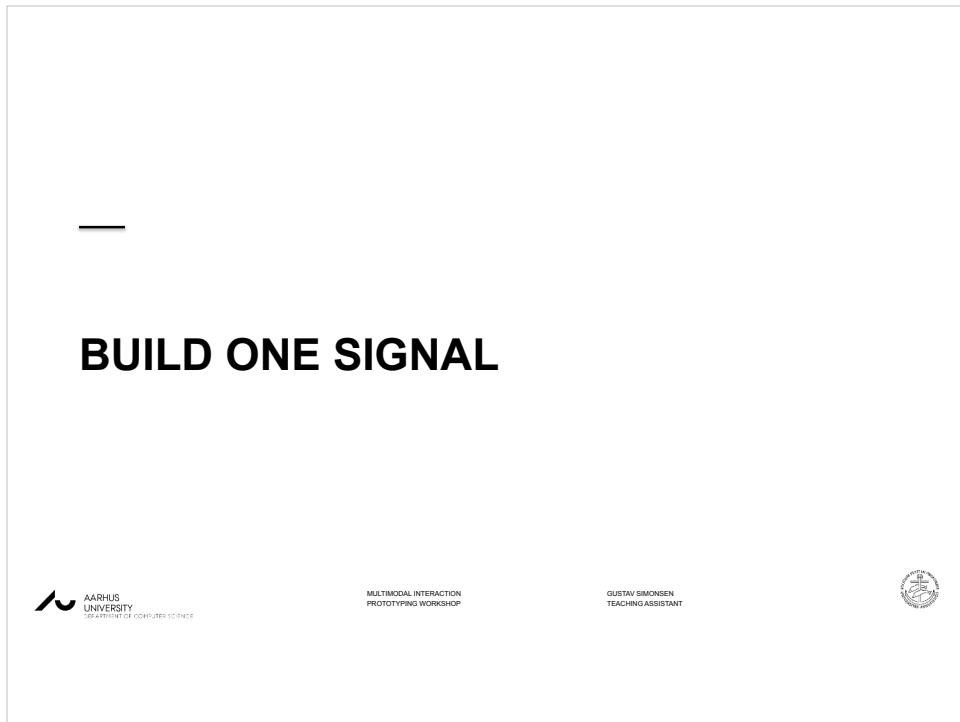


Five minutes, and put the actual repos on screen if you can.

The argument: this was built by students at your stage, not by a lab with a budget. The gap between "we should use touch" and "it moves when you touch it" is smaller than it looks.

The moving screen has a browser control panel over the serial port, which is what you will wish you had at the project demonstrations.

Note for you: the sock repo has a hardcoded Wi-Fi password and live patient IDs



1:35. The blind test.

Most groups will already have something from task 05. This is where it gets tested by someone who did not build it.

THE BRIEF

30 minutes. Not long enough to be precious about it.

One actuator, whichever you built

- Three messages: arrived, something wrong, finished

Vary rhythm above all

- Piezo: rhythm is all you have. Motor: strength too

Blind test: they name all three

- Note which two got confused, and why



Read the brief out once, then get out of the way. Full wording, since the slide is only the bones:

"Pick one actuator. Encode three messages: arrived, something wrong, finished. Vary only the rhythm of the pulses and how strong they are. Then hand it to another group with the screen turned away and see if they can name all three."

If someone asks to add a second actuator, say no. The constraint is the exercise.

THE CONSTRAINT THAT MATTERS

The recipient cannot look at the device.

If it only works while someone watches a screen,
you built a visual interface with a motor glued to it.



The line worth repeating twice.

Run the test: swap groups, receiver looks away, guess all three.

Then have each group name which two got confused and what would have separated them. That confusion is the actual learning, not the successful ones.

BEFORE YOU CALL IT DONE

Hand your device to another group with the screen turned away.

- They can tell all three messages apart
- It does not fire continuously when held
- You know which two got confused, and why

What you actually leave with

- What things feel like, how to drive one, and which to pick
- That last one is the expensive mistake this prevents



Put this up while the blind tests run, so groups can check themselves without asking you.

The middle line matters most: a device that fires continuously when held is the single most common bug, and it is a debounce problem.

Then close on the bottom half, thirty seconds. If they leave with one thing, make it the last point: pick the actuator before designing the interaction.

Discovering in week 46 that the chosen one cannot do the job is the expensive mistake this session exists to prevent



Point them at the repo. Every task sketch is commented for them, and the bench test sketches are there too if they want to see the minimal version.

Last thing to say: pick your actuator before you design the interaction, not after. Today was so they can do that from experience rather than a datasheet.

Pack down: everything back in the box, including the resistors. It will not all come back.

If anyone wants to borrow parts for the project weeks, now is when they ask